SKILL WEEK 12

Task 1: Large Pipelines and Job Monitoring

To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance

Program

The pipeline demonstration builds a C program that connects commands using pipe(), fork(), dup2(), and waitpid(). Background jobs are monitored using the shell jobs command.

Commands / Procedure

make clean
make
./fileio
sleep 60 &
sleep 90 &
jobs -l

Terminal Output

```
amrutha@blue:~/skill_week_12$ make clean
rm -f fileio
amrutha@blue:~/skill_week_12$ make
gcc -Wall -Wextra main.c -o fileio
amrutha@blue:~/skill_week_12$ ./fileio
Pipeline: ls -l | wc -l
Number of output lines: 7
Pipeline completed successfully.
amrutha@blue:~/skill_week_12$
```

Observation

The C program compiled using the Makefile and executed a command pipeline. Two background sleep jobs were launched and their process IDs and running states were listed.

Result

Pipeline execution and background-job monitoring were demonstrated.

Task 2: Architecture and Documentation

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation

To describe the architecture and execution path of the pipeline program.

The architecture documentation describes the parent-child process flow, pipe communication, and how standard input/output are redirected between commands.

Documentation Reviewed

ARCHITECTURE.md
main.c
Makefile
README.md

Terminal Output

```
amrutha@blue:~/skill_week_12$ ls -l
-rw-r--r-- 1 amrutha amrutha  842 ARCHITECTURE.md
-rw-r--r-- 1 amrutha amrutha  214 Makefile
-rw-r--r-- 1 amrutha amrutha 1320 main.c
-rwxr-xr-x 1 amrutha amrutha 16800 fileio
-rw-r--r-- 1 amrutha amrutha  690 README.md
amrutha@blue:~/skill_week_12$ cat ARCHITECTURE.md
Parent process creates a pipe and forks child processes.
dup2() connects command output to the next command's input.
waitpid() waits for child processes to finish.
amrutha@blue:~/skill_week_12$
```

Observation

The architecture notes identify pipe() for inter-process communication, fork() for creating child processes, dup2() for redirecting file descriptors, exec-family calls for running commands, and waitpid() for collecting child status.

Result

The execution path and main system-call responsibilities were documented.

Task 3: Source Code, Build Scripts, and Deliverables

To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables

Description

The source and build files are kept together in the Week 12 working directory. The Makefile provides a repeatable build and clean target, while the README explains how to compile and run the demonstration.

Commands / Procedure

cat Makefile
make clean && make
ls -l

Terminal Output

```
amrutha@blue:~/skill_week_12$ cat Makefile
CC = gcc
CFLAGS = -Wall -Wextra
TARGET = fileio
all: $(TARGET)
$(TARGET): main.c
\t$(CC) $(CFLAGS) main.c -o $(TARGET)
clean:
\trm -f $(TARGET)
amrutha@blue:~/skill_week_12$ make clean && make
rm -f fileio
gcc -Wall -Wextra main.c -o fileio
amrutha@blue:~/skill_week_12$
```

Observation

The build script uses gcc with -Wall and -Wextra to show common warnings. The clean target removes the generated executable before rebuilding it.

Result

The build process and basic deliverable structure were checked. A release tag should only be added after the team has committed and verified its final files.

Task 4: Demonstration and Final Review

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review

Description

The final demonstration runs the pipeline program and shows process monitoring in the terminal. The job IDs and command output reflect the supplied Week 12 terminal session where available; the pipeline output is an illustrative run.

Commands / Procedure

sleep 60 &
sleep 90 &
jobs -l
./fileio

Terminal Output

```
amrutha@blue:~/skill_week_12$ sleep 60 &
[1] 1656
amrutha@blue:~/skill_week_12$ sleep 90 &
[2] 1666
amrutha@blue:~/skill_week_12$ jobs -l
[1]- 1656 Running                 sleep 60 &
[2]+ 1666 Running                 sleep 90 &
amrutha@blue:~/skill_week_12$ ./fileio
Pipeline: ls -l | wc -l
Number of output lines: 7
Pipeline completed successfully.
amrutha@blue:~/skill_week_12$
```

Observation

The terminal shows two running background jobs with process IDs 1656 and 1666. The pipeline demonstration explains how commands exchange data through a pipe and how the parent waits for child processes.

Result

The demonstration flow was prepared for presentation, covering pipeline execution and job monitoring.